

ULTIMATE TECHNICAL INTERVIEW HANDBOOK

Top 100 SQL Interview Questions & Machine Coding Round Playbook

From Core Relational Theory & Window Functions to Real-Time Production Problem Solving

100

COMPREHENSIVE QUESTIONS

20

MACHINE CODING SCENARIOS

100%

OPTIMIZED SQL SOLUTIONS

Structured Curriculum & Breakdown:**Part 1:** Architecture, Execution Lifecycle, DDL/DML/DCL/TCL (Q1 – Q15)**Part 2:** Joins, Subqueries, CTEs & Set Operations (Q16 – Q30)**Part 3:** Window Functions, Ranking & Complex Analytics (Q31 – Q45)**Part 4:** Indexing, Query Optimization & Execution Plans (Q46 – Q60)**Part 5:** Transactions, ACID, Locks & Concurrency Control (Q61 – Q70)**Part 6:** Database Schema Design & Normalization (Q71 – Q80)**Part 7:** Machine Coding & Live Coding Round Problems with Solutions (Q81 – Q100)

Part 1: SQL Architecture, Order of Execution & Fundamentals (Q1 – Q15)

Q1 What is the exact logical execution order of an SQL query?

Why can't you use a column alias defined in SELECT inside a WHERE clause?

The SQL logical execution order operates in 6 core phases:

1. **FROM / JOIN:** Base tables evaluated, Cartesian products generated, ON filters applied.
2. **WHERE:** Row-level predicate filtering before grouping.
3. **GROUP BY:** Aggregation buckets formed based on unique combinations.
4. **HAVING:** Filters evaluated against aggregated group results.
5. **SELECT:** Column projections, expressions computed, window functions evaluated, column aliases created.
6. **DISTINCT:** Duplicate removal across projected rows.
7. **ORDER BY:** Sort order applied (can use aliases).
8. **LIMIT / OFFSET:** Row count slices returned.

Reason: Since WHERE is evaluated in step 2 and SELECT aliases are created in step 5, the alias does not exist yet when WHERE is parsed.

Q2 Differences between DELETE, TRUNCATE, and DROP?

Feature	DELETE	TRUNCATE	DROP
Type	DML (Data Manipulation)	DDL (Data Definition)	DDL
Scope	Specified rows (or all)	All rows only	Entire table schema + data
Rollback	Yes (logged per row)	Yes in Postgres/SQL Server (in TX), No in MySQL prior to 8.0 DDL locks	Depends on RDBMS DDL transaction support
Speed	Slow (generates transaction logs per row)	Very fast (deallocates data pages)	Instantaneous schema removal
Triggers	Fires row-level triggers	Does not fire triggers	Does not fire triggers
High Water Mark	Keeps HWM intact	Resets HWM & Auto-increment counter	Deallocates memory blocks

Q3 How does NULL evaluate in SQL Boolean logic, and what is Three-Valued Logic (3VL)?

SQL uses Three-Valued Logic: TRUE, FALSE, and UNKNOWN. Any standard comparison with NULL (e.g. `salary = NULL` or `salary != NULL`) evaluates to UNKNOWN. In a WHERE clause, rows are only returned if the predicate evaluates to TRUE (UNKNOWN is discarded).

Key Rules: NULL = NULL is UNKNOWN; NULL IS NULL is TRUE; NOT (UNKNOWN) is UNKNOWN. In aggregations, COUNT(col) ignores NULLs while COUNT(*) includes rows with NULLs.

Q4 Difference between WHERE and HAVING clauses with performance implications?

WHERE: Filters individual rows *before* aggregation takes place. It utilizes indexes on raw table columns efficiently to reduce row sets.

HAVING: Filters aggregated grouped rows *after* the GROUP BY stage. It cannot use raw b-tree indexes unless operating on the group key.

Rule: Always push non-aggregate filters into WHERE to minimize memory footprint during grouping.

Q5 Categorize SQL Commands: DDL, DML, DCL, TCL, and DQL.

- **DDL (Data Definition Language):** CREATE, ALTER, DROP, TRUNCATE, RENAME.
- **DML (Data Manipulation Language):** INSERT, UPDATE, DELETE, MERGE / UPSERT.
- **DQL (Data Query Language):** SELECT.
- **DCL (Data Control Language):** GRANT, REVOKE.
- **TCL (Transaction Control Language):** COMMIT, ROLLBACK, SAVEPOINT, SET TRANSACTION.

Q6 How do CHAR, VARCHAR, and NVARCHAR differ in storage and padding?

CHAR(*n*) stores fixed-length character strings, right-padded with spaces up to *n* bytes (faster for predictable lengths like UUIDs or country codes). VARCHAR(*n*) stores variable-length strings with 1–2 byte prefix headers indicating length (space-efficient). NVARCHAR(*n*) stores variable-length Unicode data (UTF-16/UCS-2), consuming 2 bytes per character to support multi-language characters.

Q7 What are Primary Keys vs. Unique Keys vs. Super Keys vs. Candidate Keys?

- **Super Key:** Any set of columns that uniquely identifies a record.
- **Candidate Key:** A minimal Super Key with no redundant attributes.
- **Primary Key:** The chosen Candidate Key; enforces uniqueness + strict NOT NULL; typically creates a Clustered Index.
- **Unique Key:** Enforces uniqueness; allows one (or multiple depending on RDBMS) NULL values; typically creates a Non-Clustered Index.

Q8 What are Foreign Key Constraints and Referential Actions (Cascade, Set Null, Restrict)?

A Foreign Key links a child table column to a parent table's Primary/Unique key to maintain referential integrity. Actions on parent row deletion/update:

- ON DELETE CASCADE: Deleting parent deletes all corresponding child rows.
- ON DELETE SET NULL: Deleting parent sets foreign key in child rows to NULL.
- ON DELETE RESTRICT / NO ACTION: Rejects parent deletion if dependent child records exist.

Q9 What is the difference between COUNT(*), COUNT(1), and COUNT(column_name)?

COUNT(*) and COUNT(1) are semantically and practically identical in modern query optimizers—they count total rows in the result set regardless of NULLs. COUNT(column_name) iterates through rows and counts only non-null values for that specific column.

Q10 Explain COALESCE() vs ISNULL() vs NVL() vs IFNULL().

COALESCE(val1, val2, ..., valN) is ANSI SQL standard; accepts *n* arguments and returns the first non-NULL value. ISNULL(col, replacement) is T-SQL specific (2 arguments). NVL(col, replacement) is Oracle specific. IFNULL(col, replacement) is MySQL specific.

Q11 How does EXISTS differ from IN when querying subqueries with NULLs?

EXISTS returns TRUE as soon as a single match is found (short-circuit boolean evaluation). IN unpacks the subquery into a set of values: val IN (v1, v2, NULL). **Critical Bug:** NOT IN evaluates to UNKNOWN (returning 0 rows) if the subquery contains even a single NULL! NOT EXISTS does not suffer from this issue.

Q12 What is the difference between UNION and UNION ALL?

UNION combines datasets and executes a distinct sort/hash operation to deduplicate rows, which is CPU and memory intensive. UNION ALL concatenates datasets directly without deduplication. Always default to UNION ALL unless deduplication is explicitly required.

Q13 Explain the difference between Stored Procedures and User-Defined Functions (UDFs).

- **Functions:** Must return a value, cannot execute DDL/DML (read-only), can be embedded inside SELECT/WHERE clauses.
- **Stored Procedures:** Can return 0 or multiple result sets/output parameters, can execute DDL/DML and manage transactions (COMMIT/ROLLBACK), invoked with CALL or EXEC.

Q14 What are Temporary Tables vs Common Table Expressions (CTEs) vs Views?

- **View:** Saved query definition; no physical storage (unless indexed/materialized); re-executed every time referenced.
- **CTE:** Query-scoped named result set; aids readability; in standard SQL, ephemeral during statement execution.
- **Temp Table:** Physical table written to tempdb/memory; supports indexes, statistics, and persists across multiple queries within a session.

Q15 How does GROUP BY GROUPING SETS, ROLLUP, and CUBE work?

Used for multi-dimensional OLAP aggregations in a single query:

- **ROLLUP (a, b):** Generates hierarchical totals: (a, b), (a), and grand total ().
- **CUBE (a, b):** Generates all 2^N permutations: (a, b), (a), (b), and ().
- **GROUPING SETS ((a), (b)):** Explicitly defines exact aggregation combinations without unwanted combinations.

Part 2: Joins, Subqueries & Complex Data Manipulation (Q16 – Q30)

Q16 Visual and Mathematical Breakdown of All SQL Join Types.

- **INNER JOIN:** Returns rows when matching keys exist in both tables.
- **LEFT (OUTER) JOIN:** All rows from left table + matched rows from right table (NULL for unmatched).
- **RIGHT (OUTER) JOIN:** All rows from right table + matched rows from left table.
- **FULL OUTER JOIN:** All rows from both tables; unlinked sides filled with NULL.
- **CROSS JOIN:** Cartesian product ($M \times N$ rows).
- **SELF JOIN:** Joining a table with itself (e.g., Employee to Manager hierarchy).

Q17 What happens when joining on non-unique keys (Fan-Out problem)?

If Table A has 3 rows with key = 'X' and Table B has 4 rows with key = 'X', an **INNER JOIN** generates $3 \times 4 = 12$ rows. This unexpected Cartesian multiplication is known as **Join Fan-out** and causes severe calculation bugs in aggregate functions (e.g., inflated `SUM(order_amount)`). Solution: Pre-aggregate child tables before joining.

Q18 Find all employees who earn more than their direct managers (Self Join).

```
SELECT e.emp_name AS employee, e.salary, m.emp_name AS manager, m.salary AS mgr_salary
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Q19 Difference between Filter in ON clause vs. Filter in WHERE clause in LEFT JOIN.

In a **LEFT JOIN**:

- **LEFT JOIN B ON A.id = B.id AND B.status = 'ACTIVE':** Returns ALL rows from A. For un-matched or non-active B rows, B's columns are NULL.
- **LEFT JOIN B ON A.id = B.id WHERE B.status = 'ACTIVE':** Evaluates *after* the join, effectively converting the **LEFT JOIN** into an **INNER JOIN** because NULL status rows are filtered out!

Q20 What is a Correlated Subquery vs Non-Correlated Subquery?

Non-Correlated: Executes once independently of the outer query; optimizer caches results (e.g., `WHERE salary > (SELECT AVG(salary) FROM emp)`).

Correlated: References columns from the outer query row; evaluated once per row processed by the outer query unless converted into an index join by the optimizer.

Q21 Write a query to find employees earning above the average salary of their department.

```
-- Method: Correlated Subquery or Window Function
SELECT emp_id, emp_name, dept_id, salary
FROM (
    SELECT emp_id, emp_name, dept_id, salary,
           AVG(salary) OVER (PARTITION BY dept_id) AS avg_dept_sal
    FROM employees
) t
WHERE salary > avg_dept_sal;
```

Q22 How to perform Anti-Joins: Find customers who never placed an order.

```
-- Approach 1: LEFT JOIN with IS NULL (Best compatibility)
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

-- Approach 2: NOT EXISTS (Highly optimized in PostgreSQL / Oracle)
SELECT c.customer_id, c.name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

Q23 What is a Cross Apply / Lateral Join?

A LATERAL join (PostgreSQL/Oracle) or CROSS APPLY (SQL Server) acts like a for-each loop in SQL. It allows a subquery/table function on the right-hand side to reference columns produced by the left-hand side table row-by-row.

Q24 Get Top 2 highest priced products for each category using LATERAL JOIN.

```
SELECT c.category_name, p.product_name, p.price
FROM categories c
CROSS JOIN LATERAL (
    SELECT product_name, price
    FROM products
    WHERE category_id = c.category_id
    ORDER BY price DESC
    LIMIT 2
) p;
```

Q25 Explain Recursive CTEs and write a query for organizational hierarchy level traversal.

```
WITH RECURSIVE OrgHierarchy AS (
    -- Anchor Member: Top-level CEO
    SELECT emp_id, emp_name, manager_id, 1 AS depth_level
    FROM employees
    WHERE manager_id IS NULL
    UNION ALL
    -- Recursive Member: Subordinates
    SELECT e.emp_id, e.emp_name, e.manager_id, h.depth_level + 1
    FROM employees e
    JOIN OrgHierarchy h ON e.manager_id = h.emp_id
)
SELECT * FROM OrgHierarchy ORDER BY depth_level, emp_id;
```

Q26 Find duplicate rows in a table without using window functions.

```
SELECT email, COUNT(*) AS occurrence_count
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Q27 Delete duplicate records keeping only the lowest id.

```
DELETE FROM users
WHERE id NOT IN (
    SELECT min_id FROM (
        SELECT MIN(id) AS min_id
        FROM users
        GROUP BY email
    ) AS keep_rows
);
```

Q28 How to implement Pivot and Unpivot operations?

Pivot: Rotating row values into separate columns (using conditional aggregation SUM(CASE WHEN ...)).

Unpivot: Transposing columns into rows (using UNION ALL or CROSS JOIN LATERAL / UNNEST).

Q29 Pivot Monthly Sales: Convert months to columns.

```
SELECT
    emp_id,
    SUM(CASE WHEN sale_month = 'Jan' THEN amount ELSE 0 END) AS Jan_Sales,
    SUM(CASE WHEN sale_month = 'Feb' THEN amount ELSE 0 END) AS Feb_Sales,
    SUM(CASE WHEN sale_month = 'Mar' THEN amount ELSE 0 END) AS Mar_Sales
FROM monthly_sales
GROUP BY emp_id;
```

Q30 Difference between INTERSECT, EXCEPT / MINUS, and UNION.

Set operators require identical column count and compatible data types:

- **INTERSECT:** Returns only distinct rows common to both query sets.
- **EXCEPT / MINUS:** Returns distinct rows present in the first set but absent in the second set.

Part 3: Window Functions & Advanced Analytical Queries (Q31 – Q45)

Q31 Detailed Comparison: ROW_NUMBER() vs RANK() vs DENSE_RANK().

Suppose salaries are: [100, 90, 90, 80]:

Salary	ROW_NUMBER()	RANK()	DENSE_RANK()
100	1	1	1
90	2	2	2
90	3	2	2
80	4	4 (skips 3)	3 (no gaps)

Use Case: Use DENSE_RANK() when finding the Nth highest salary to handle ties properly without skipping rank positions.

Q32 Find the N-th Highest Salary in an Organization (Handling duplicates).

```
WITH RankedSalaries AS (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
)
SELECT salary FROM RankedSalaries WHERE rnk = 3 LIMIT 1;
```

Q33 Calculate 7-Day Moving Average of Daily Revenue.

```
SELECT
    order_date,
    daily_revenue,
    AVG(daily_revenue) OVER (
        ORDER BY order_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS moving_avg_7d
FROM daily_revenue_summary;
```

Q34 Difference between ROWS BETWEEN and RANGE BETWEEN in window frame specification.

ROWS operates on physical row offsets regardless of value duplicates (e.g., exactly 2 physical rows above). RANGE operates on logical value differences based on the ORDER BY column (e.g., all rows with the same timestamp or within an interval of 7 days).

Q35 Calculate Year-over-Year (YoY) Growth percentage using LAG().

```
WITH YearlySales AS (
    SELECT
        EXTRACT(YEAR FROM order_date) AS yr,
        SUM(amount) AS total_sales
    FROM orders
    GROUP BY 1
)
SELECT
    yr,
    total_sales,
    LAG(total_sales, 1) OVER (ORDER BY yr) AS prev_year_sales,
    ROUND(100.0 * (total_sales - LAG(total_sales, 1) OVER (ORDER BY yr))
        / LAG(total_sales, 1) OVER (ORDER BY yr), 2) AS yoy_pct_growth
FROM YearlySales;
```

Q36 Explain LEAD() vs LAG() and how to provide default fallback values.

LAG(col, offset, default) accesses preceding rows; LEAD(col, offset, default) accesses following rows. Example:
LAG(price, 1, 0) OVER (ORDER BY date) returns 0 instead of NULL when no previous row exists.

Q37 Explain FIRST_VALUE(), LAST_VALUE(), and NTH_VALUE() gotchas.

Gotcha: The default window frame for ORDER BY is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. Because of this, LAST_VALUE() will return the current row instead of the partition's final row unless you explicitly specify: ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.

Q38 Compute Running Cumulative Total with Partition Reset.

```
SELECT
    dept_id,
    emp_name,
    salary,
    SUM(salary) OVER (
        PARTITION BY dept_id
        ORDER BY emp_id
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_dept_salary
FROM employees;
```

Q39 What is NTILE(n) and how does it distribute non-divisible rows?

NTILE(n) divides sorted rows into *n* buckets as equally as possible. If the count is not evenly divisible, extra rows are distributed to earlier buckets first (e.g. 10 rows into 4 buckets gives bucket sizes: 3, 3, 2, 2).

Q40 Categorize users into Quartiles based on lifetime spend.

```
SELECT
    user_id,
    total_spent,
    NTILE(4) OVER (ORDER BY total_spent DESC) AS spend_quartile
FROM user_lifetime_value;
```

Q41 What is PERCENT_RANK() vs CUME_DIST()?

PERCENT_RANK() = (rank - 1) / (total_rows - 1); returns relative rank between 0 and 1.

CUME_DIST() = (count of rows with values <= current row value) / total_rows; returns cumulative distribution percentage (0 < cume_dist <= 1).

Q42 Identify Consecutive Active Days (Gaps and Islands Problem).

Classic algorithmic technique: Subtracting a sequential ROW_NUMBER() from the consecutive date produces a constant grouping key (Island ID) for consecutive streaks.

```
WITH RankedLogins AS (
    SELECT DISTINCT user_id, login_date,
        login_date - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date)::INTEGER AS grp
    FROM user_logins
)
SELECT user_id, MIN(login_date) AS streak_start, MAX(login_date) AS streak_end, COUNT(*) AS streak_days
FROM RankedLogins
GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
```

Q43 Find the Longest Continuous Inactive Streak per Account (Gaps problem).

```
WITH NextTx AS (
    SELECT account_id, tx_date,
        LEAD(tx_date) OVER (PARTITION BY account_id ORDER BY tx_date) AS next_date
    FROM transactions
)
SELECT account_id, MAX(next_date - tx_date) AS max_inactivity_gap
FROM NextTx
WHERE next_date IS NOT NULL
GROUP BY account_id;
```


Q44 Conditionally fill missing forward values (Forward Fill in SQL).

```
WITH ValueGroup AS (
    SELECT id, record_date, metric_val,
           COUNT(metric_val) OVER (ORDER BY record_date) AS val_partition
    FROM sensor_readings
)
SELECT id, record_date,
       FIRST_VALUE(metric_val) OVER (
           PARTITION BY val_partition
           ORDER BY record_date
       ) AS ffilled_val
FROM ValueGroup;
```

Q45 Calculate Market Basket Affinity / Co-occurrence count.

```
SELECT
    o1.product_id AS product_a,
    o2.product_id AS product_b,
    COUNT(*) AS bought_together_frequency
FROM order_items o1
JOIN order_items o2
    ON o1.order_id = o2.order_id
   AND o1.product_id < o2.product_id
GROUP BY o1.product_id, o2.product_id
ORDER BY bought_together_frequency DESC
LIMIT 10;
```

Part 4: Indexing, Query Optimization & Execution Plans (Q46 – Q60)

Q46 Difference between Clustered Index and Non-Clustered Index?

Clustered Index: Dictates physical order of data on disk. Only ONE clustered index per table. Leaf nodes contain the actual data rows.

Non-Clustered Index: Separate B-tree structure pointing to the physical data rows (via Row ID or Clustered Key). Multiple non-clustered indexes allowed per table.

Q47 What is a Covering Index and how does it prevent Bookmark/Key Lookups?

A Covering Index contains all columns requested in a query (both filtered columns in keys and selected columns via INCLUDE (col1, col2)). When an index covers a query, the RDBMS engine retrieves the full result set entirely from the index leaf pages without performing random I/O read lookups to the base table heap/clustered index.

Q48 What is Index Cardinality and how does Column Selectivity dictate index utility?

Selectivity = Distinct Values / Total Rows. High selectivity (e.g., user_uuid near 1.0) is ideal for B-tree indexing. Low selectivity (e.g., gender with 2 values across 10M rows) causes optimizer to ignore index and perform a Full Table Scan because sequential block scan is faster than thousands of random I/O index lookups.

Q49 Why does WHERE YEAR(created_at) = 2024 break index usage (SARGability)?

Applying functions or mathematical operations on indexed columns (e.g., YEAR(col), UPPER(col), col + 10) makes the predicate **Non-SARGable** (Search Argument Able). The B-tree cannot binary search calculated expressions.

Fix: Rewrite to range: WHERE created_at >= '2024-01-01' AND created_at < '2025-01-01'.

Q50 What is the Leftmost Prefix Rule in Composite Indexes?

For a composite index on (ColA, ColB, ColC):

- Queries filtering on (ColA) or (ColA, ColB) or (ColA, ColB, ColC) USE the index.
- Queries filtering on (ColB) or (ColB, ColC) CANNOT use the B-tree index tree search (only index scan).
- If a range condition (e.g. ColB > 10) occurs, subsequent columns (ColC) cannot be used for index seeking.

Q51 How do B-Tree Indexes work internally vs Hash Indexes vs GIN / GiST Indexes?

- **B-Tree Index:** Balanced tree; supports exact matches, range scans (BETWEEN, <, >), prefixes, and ordered traversal.
- **Hash Index:** O(1) exact lookups only; does not support range scans or sorting.
- **GIN (Generalized Inverted Index):** Used for array elements, full-text search, and JSONB document indexing.

Q52 Explain how to read and interpret an EXPLAIN ANALYZE query execution plan.

Key indicators to inspect in execution plans:

- **Seq Scan / Full Table Scan:** Reading entire table disk blocks (check for missing index or low selectivity).
- **Index Seek vs Index Scan:** Index Seek traverses B-tree directly to target leaf (fast); Index Scan reads entire index.
- **Actual Rows vs Estimated Rows:** Large discrepancies indicate outdated table statistics; run ANALYZE table_name.
- **Join Algorithms:** Nested Loop (best for small outer, indexed inner), Hash Join (large unsorted sets), Merge Join (large pre-sorted sets).

Q53 How do Hash Join, Nested Loop Join, and Merge Join differ in performance?

- **Nested Loop Join:** Outer loop scans rows, inner loop searches index. Complexity: O(N log M). Fast for tiny outer datasets.
- **Hash Join:** Builds in-memory hash table on smaller relation, probes with larger relation. Memory intensive, ideal for large ad-hoc joins.
- **Merge Join:** Requires both tables sorted on join keys; scans both pointers forward. Fast for massive sorted/indexed datasets.

Q54 Why are wildcard queries like LIKE '%apple%' slow and how do you optimize them?

Leading wildcards (%apple) bypass standard B-tree index search because the prefix is unknown.

Optimization: Use Full-Text Search (tsvector/MATCH AGAINST), Trigram Indexes (PostgreSQL pg_trgm with GIN index), or reverse string indexing for suffix lookups.

Q55 What is Database Partitioning (Range, List, Hash) vs Sharding?

Partitioning: Dividing a single table's data across multiple physical storage segments within the *same database instance* (e.g., Partition by Year). Enables Partition Pruning.

Sharding: Horizontal distribution of database rows across multiple *independent server instances / nodes* across a cluster.

Q56 What is Partition Pruning?

Optimizer optimization where the query engine examines WHERE predicates and skips scanning partitions that cannot contain matching rows (e.g., query for 2024-05 only scans the orders_2024 partition, eliminating 90%+ I/O reads).

Q57 What is Keyset Pagination (Seek Method) vs OFFSET / LIMIT?

OFFSET 1000000 LIMIT 20 forces database engine to read 1,000,020 rows from disk and discard 1,000,000 (O(N) latency).

Keyset Pagination: Uses index seeking: WHERE id > last_seen_id ORDER BY id ASC LIMIT 20 (O(1) constant time latency regardless of depth).

Q58 What is Table Bloat and why is VACUUM needed in PostgreSQL?

PostgreSQL implements Multi-Version Concurrency Control (MVCC) by writing new row versions on UPDATE and tagging old rows as dead tuples on DELETE. VACUUM reclaims space occupied by dead tuples and prevents transaction ID wraparound. VACUUM FULL rewrites the entire table to release disk space back to the OS.

Q59 How do Partial / Filtered Indexes optimize storage and write overhead?

A partial index indexes only a subset of table rows matching a condition:

```
CREATE INDEX idx_unprocessed_orders ON orders(id) WHERE status = 'PENDING';
```

Benefits: Saves 95%+ disk/RAM footprint and prevents index maintenance overhead on completed rows.

Q60 What are Database Statistics, Histograms, and why does Stale Statistics cause bad query plans?

The Cost-Based Optimizer (CBO) uses column data distribution histograms to estimate cardinality and pick execution algorithms. Stale stats cause the optimizer to choose slow Nested Loop scans over Hash Joins or perform sequential scans instead of index seeks.

Part 5: Transactions, ACID, Concurrency & Locking (Q61 – Q70)

Q61 Explain ACID properties in depth.

- **Atomicity:** All operations succeed or entire transaction is rolled back (via Write-Ahead Logging / Undo Logs).
- **Consistency:** Data moves from one valid state to another, preserving all schema constraints and foreign keys.
- **Isolation:** Concurrent transactions execute without cross-contamination.
- **Durability:** Committed transactions persist even through power loss or system crashes (flushed to WAL disk).

Q62 Breakdown of the 4 ANSI SQL Transaction Isolation Levels and Anomalies.

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Allowed	Allowed	Allowed
Read Committed	Prevented	Allowed	Allowed
Repeatable Read	Prevented	Prevented	Allowed (prevented in Postgres MVCC / MySQL InnoDB Next-Key Locks)
Serializable	Prevented	Prevented	Prevented

Q63 Define Read Anomalies: Dirty Read, Non-Repeatable Read, Phantom Read, and Serialization Anomaly.

- **Dirty Read:** Tx 1 reads uncommitted changes from Tx 2 (which later rolls back).
- **Non-Repeatable Read:** Tx 1 reads a row; Tx 2 updates/deletes that row; Tx 1 re-reads and sees altered data.
- **Phantom Read:** Tx 1 queries a range of rows; Tx 2 inserts a new row matching that range; Tx 1 re-queries and sees "phantom" new rows.

Q64 What is Optimistic Concurrency Control (OCC) vs Pessimistic Concurrency Control (PCC)?

Pessimistic: Locks rows immediately upon read (`SELECT ... FOR UPDATE`) to block other writers until commit. High safety, lower concurrency.

Optimistic: Reads without locking; uses a `version_number` or timestamp. On update: `UPDATE tbl SET col = val, version = version + 1 WHERE id = 1 AND version = current_version`. If row count is 0, aborts and retries. High throughput for low-conflict workloads.

Q65 How does SELECT ... FOR UPDATE work and when should you use SKIP LOCKED?

`SELECT ... FOR UPDATE` places exclusive row-level locks on selected records.

`SKIP LOCKED` skips rows already locked by concurrent workers. Essential for building high-throughput transactional **Message Queue / Job Worker systems** in SQL without lock contention bottlenecks.

Q66 What is a Deadlock, how do engines detect it, and how do you prevent it in application code?

A Deadlock occurs when Tx 1 holds Lock A and waits for Lock B, while Tx 2 holds Lock B and waits for Lock A (circular dependency). RDBMS runs background deadlock detection graph algorithms and aborts the transaction with lowest cost.

Prevention: Always acquire locks in the exact same deterministic order across all application endpoints; keep transactions short.

Q67 What is Write-Ahead Logging (WAL) and Checkpointing?

WAL guarantees Durability: Changes are written sequentially to append-only log files on disk *before* dirty data pages are flushed from RAM buffer cache to base tables.

Checkpoint: Flushes all in-memory dirty buffer pool pages to disk and advances the safe WAL recovery pointer.

Q68 Shared Locks (S-Locks) vs Exclusive Locks (X-Locks) vs Intent Locks.

- **Shared Lock (S):** Multiple readers can hold S-locks concurrently; blocks X-locks.
- **Exclusive Lock (X):** Single transaction holds exclusive write access; blocks all other S and X locks.
- **Intent Lock (IS / IX):** Table-level lock indicating that a transaction intends to acquire fine-grained row locks below, avoiding full table scans during locking.

Q69 How does MVCC (Multi-Version Concurrency Control) eliminate read-write blocking?

MVCC ensures "*Readers never block Writers, and Writers never block Readers.*" Each row maintains creation and expiration transaction timestamps (e.g. xmin / xmax in PostgreSQL). Readers view a consistent snapshot corresponding to their transaction start time without locking records.

Q70 What is Two-Phase Commit (2PC) in Distributed Databases?

Coordinator protocol for atomic transactions across multiple distributed database nodes:

1. **Prepare Phase:** Coordinator asks all participant nodes to execute and write WAL logs; nodes reply "PREPARED" or "ABORT".
2. **Commit Phase:** If all nodes agreed, coordinator issues "COMMIT"; otherwise issues "ROLLBACK".

Part 6: Database Design, Schema Modeling & Normalization (Q71 – Q80)

Q71 Normalization Forms: 1NF, 2NF, 3NF, and BCNF Explained with Real Examples.

- **1NF (First Normal Form):** Atomic values (no arrays/comma-separated strings in columns), unique primary key.
- **2NF (Second Normal Form):** In 1NF + No Partial Dependency (all non-key columns must depend on the *entire* composite primary key).
- **3NF (Third Normal Form):** In 2NF + No Transitive Dependency (non-key columns must not depend on other non-key columns: A -> B -> C).
- **BCNF (Boyce-Codd Normal Form):** Stricter 3NF; for every functional dependency X -> Y, X must be a Super Key.

Q72 When and Why should you Denormalize a Database Schema?

Denormalization introduces controlled redundancy to optimize heavy read queries and reduce costly multi-table joins in high-traffic OLAP data warehouses and reporting systems. Tradeoffs: Increased write overhead and risk of data anomalies.

Q73 OLTP (Online Transaction Processing) vs OLAP (Online Analytical Processing) Schema Design.

Dimension	OLTP	OLAP
Goal	High concurrency, fast ACID reads/writes	Heavy complex analytical aggregation
Design	Highly normalized (3NF/BCNF)	Star Schema / Snowflake Schema
Storage	Row-oriented (PostgreSQL, MySQL)	Columnar (ClickHouse, Snowflake, Redshift)

Q74 Star Schema vs Snowflake Schema in Data Warehousing.

Star Schema: Central Fact Table surrounded by completely denormalized Dimension Tables (fewer joins, fastest query performance).

Snowflake Schema: Dimension tables are normalized into sub-dimensions (saves disk space, requires more joins).

Q75 What are Slowly Changing Dimensions (SCD Type 1, Type 2, Type 3)?

- **SCD Type 1:** Overwrite old value with new value (no historical tracking).
- **SCD Type 2:** Add new record with validity date ranges (`start_date`, `end_date`, `is_current` flag) for full audit history.
- **SCD Type 3:** Keep previous value in a dedicated column (`current_dept`, `previous_dept`).

Q76 How to design Self-Referencing / Adjacency List vs Nested Sets for Hierarchical Data?

Adjacency List (`parent_id`): Simple schema, straightforward inserts, requires Recursive CTEs to query.

Closure Table: Separate table storing every ancestor-descendant path and depth. Fast subtree lookups at the cost of extra join table writes.

Q77 UUID (v4 / v7) vs Auto-Increment BIGINT as Primary Keys.

BIGINT AUTO_INCREMENT: 8 bytes, sequential, perfect B-tree clustering, leakable sequential ID security risk. **UUID v4:** 16 bytes, completely random, destroys B-tree page locality causing severe index page splits. **UUID v7:** 16 bytes, **time-ordered** UUID combining Unix epoch prefix with random bits — ideal for distributed systems while maintaining B-tree index insertion efficiency.

Q78 How to handle Soft Deletes (`deleted_at`) and solve Unique Constraint conflicts?

When soft deleting (`is_deleted = TRUE`), a standard `UNIQUE(email)` constraint prevents re-registration of the same email.

Solution: Use a Partial Unique Index:

```
CREATE UNIQUE INDEX uq_active_user_email ON users(email) WHERE deleted_at IS NULL;
```

Q79 Polymorphic Associations vs Concrete Foreign Keys.

Polymorphic associations store `parent_type` ('POST', 'COMMENT') and `parent_id` in one table, breaking true database-level Foreign Key constraints and referential integrity.

Best Practice: Use concrete nullable FKs (`post_id REFERENCES posts`, `comment_id REFERENCES comments`) with a check constraint ensuring exactly one is not null.

Q80 Relational JSON/JSONB Data Modeling: When to use JSON columns vs Normalized Tables?

Use JSONB for dynamic, sparse, highly variable schema attributes (e.g. third-party webhook payloads, user UI preferences). Use normalized relational columns for relational foreign keys, core business entities, frequently filtered attributes, and financial balances.

Part 7: Machine Coding Round — 20 Practical Scenarios with Code (Q81 – Q100)

Q81 - MACHINE ROUND E-Commerce High-Value Customer Retention: Top 3 Spenders per Category.

Problem: Write a performant query to return the top 3 highest spending customers within each product category along with their rank and total spend.

```
WITH CustomerCategorySpend AS (
    SELECT
        p.category_id,
        o.customer_id,
        SUM(oi.quantity * oi.unit_price) AS total_spent,
        DENSE_RANK() OVER (
            PARTITION BY p.category_id
            ORDER BY SUM(oi.quantity * oi.unit_price) DESC
        ) AS spend_rank
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    JOIN products p ON oi.product_id = p.product_id
    WHERE o.order_status = 'COMPLETED'
    GROUP BY p.category_id, o.customer_id
)
SELECT category_id, customer_id, total_spent, spend_rank
FROM CustomerCategorySpend
WHERE spend_rank <= 3
ORDER BY category_id, spend_rank;
```

Q82 - MACHINE ROUND FinTech Fraud Detection: Rapid Successive Transactions within 5 Minutes.

Problem: Flag any user who triggered 3 or more transactions within any rolling 5-minute time window.

```
WITH TxWindow AS (
    SELECT
        tx_id,
        user_id,
        tx_time,
        amount,
        COUNT(*) OVER (
            PARTITION BY user_id
            ORDER BY tx_time
            RANGE BETWEEN INTERVAL '5 minutes' PRECEDING AND CURRENT ROW
        ) AS rolling_tx_count
    FROM transactions
)
SELECT DISTINCT user_id, tx_id, tx_time, rolling_tx_count
FROM TxWindow
WHERE rolling_tx_count >= 3;
```

Q83 - MACHINE ROUND Ride Hailing (Uber/Lyft): Driver Acceptance Rate Analysis.

Problem: Calculate daily driver dispatch acceptance rates and compare against their 30-day trailing baseline average.

```
WITH DailyStats AS (
    SELECT
        driver_id,
        dispatch_date,
        ROUND(100.0 * SUM(CASE WHEN status = 'ACCEPTED' THEN 1 ELSE 0 END) / COUNT(*), 2) AS daily_rate
    FROM driver_dispatches
    GROUP BY driver_id, dispatch_date
)
SELECT
    driver_id,
    dispatch_date,
    daily_rate,
    AVG(daily_rate) OVER (
        PARTITION BY driver_id
        ORDER BY dispatch_date
        ROWS BETWEEN 29 PRECEDING AND CURRENT ROW
    ) AS trailing_30d_avg_rate
FROM DailyStats;
```

Q84 - MACHINE ROUND SaaS User Retention: Monthly Cohort Retention Matrix.

Problem: Build a monthly cohort retention table tracking the percentage of users returning in Month 1, 2, 3 after signup.

```
WITH UserFirstMonth AS (
    SELECT user_id, DATE_TRUNC('month', MIN(created_at)) AS cohort_month
    FROM users GROUP BY user_id
),
UserActivities AS (
    SELECT
        u.cohort_month,
        EXTRACT(YEAR FROM a.activity_date) * 12 + EXTRACT(MONTH FROM a.activity_date) -
        (EXTRACT(YEAR FROM u.cohort_month) * 12 + EXTRACT(MONTH FROM u.cohort_month)) AS month_number,
        COUNT(DISTINCT a.user_id) AS active_users
    FROM UserFirstMonth u
    JOIN user_events a ON u.user_id = a.user_id
    GROUP BY 1, 2
),
CohortSizes AS (
    SELECT cohort_month, COUNT(*) AS total_users
    FROM UserFirstMonth GROUP BY cohort_month
)
SELECT
    c.cohort_month,
    c.total_users AS cohort_size,
    ua.month_number,
    ROUND(100.0 * ua.active_users / c.total_users, 2) AS retention_pct
FROM CohortSizes c
JOIN UserActivities ua ON c.cohort_month = ua.cohort_month
ORDER BY c.cohort_month, ua.month_number;
```

Q85 - MACHINE ROUND Warehouse Inventory Reconciliation: Running Balance with Safety Stock Alerts.

Problem: Track running inventory balances per SKU from receipts and dispatches and alert when balance drops below safety stock.

```
WITH RunningInventory AS (
    SELECT
        l.sku_id,
        l.event_time,
        l.movement_qty,
        SUM(l.movement_qty) OVER (
            PARTITION BY l.sku_id
            ORDER BY l.event_time
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ) AS current_stock,
        s.safety_stock_threshold
    FROM inventory_ledger l
    JOIN skus s ON l.sku_id = s.sku_id
)
SELECT *,
    CASE WHEN current_stock < safety_stock_threshold THEN 'ALERT_LOW_STOCK' ELSE 'OK' END AS
stock_status
FROM RunningInventory;
```

Q86 - MACHINE ROUND Subscription Churn Rate Calculation (Monthly Recurring).

Problem: Calculate monthly churn rate percentage: (Churned Users in Month / Active Users at Month Start) * 100.

```
WITH MonthlyStatus AS (
    SELECT
        DATE_TRUNC('month', d.calendar_date) AS month_start,
        COUNT(DISTINCT CASE WHEN s.start_date <= d.calendar_date AND (s.cancel_date IS NULL OR
s.cancel_date > d.calendar_date) THEN s.user_id END) AS start_active_users,
        COUNT(DISTINCT CASE WHEN DATE_TRUNC('month', s.cancel_date) = DATE_TRUNC('month', d.calendar_date)
THEN s.user_id END) AS churned_users
    FROM calendar_dates d
    JOIN subscriptions s ON s.start_date <= d.calendar_date
    WHERE d.calendar_date = DATE_TRUNC('month', d.calendar_date)
    GROUP BY 1
)
SELECT
    month_start,
    start_active_users,
    churned_users,
    ROUND(100.0 * churned_users / NULLIF(start_active_users, 0), 2) AS churn_rate_pct
FROM MonthlyStatus
ORDER BY month_start;
```

Q87 - MACHINE ROUND Banking Ledger Double-Entry Audit Verification.

Problem: Validate that every transaction in the ledger conforms to double-entry accounting rules (Debit total equals Credit total).

```
SELECT
    transaction_id,
    SUM(CASE WHEN entry_type = 'DEBIT' THEN amount ELSE 0 END) AS total_debit,
    SUM(CASE WHEN entry_type = 'CREDIT' THEN amount ELSE 0 END) AS total_credit
FROM journal_entries
GROUP BY transaction_id
HAVING SUM(CASE WHEN entry_type = 'DEBIT' THEN amount ELSE 0 END) !=
SUM(CASE WHEN entry_type = 'CREDIT' THEN amount ELSE 0 END);
```


Q88 - MACHINE ROUND Ad-Tech Click-Through Conversion Attribution (30-Minute Attribution Window).

Problem: Attribute conversions to the most recent ad click occurring within 30 minutes prior to checkout (Last-Touch Attribution).

```
WITH RankedClicks AS (
    SELECT
        c.conversion_id,
        c.user_id,
        c.conversion_time,
        c.revenue,
        clk.ad_id,
        clk.click_time,
        ROW_NUMBER() OVER (
            PARTITION BY c.conversion_id
            ORDER BY clk.click_time DESC
        ) AS click_rank
    FROM conversions c
    JOIN ad_clicks clk
        ON c.user_id = clk.user_id
        AND clk.click_time <= c.conversion_time
        AND clk.click_time >= c.conversion_time - INTERVAL '30 minutes'
)
SELECT conversion_id, user_id, ad_id, conversion_time, revenue
FROM RankedClicks
WHERE click_rank = 1;
```

Q89 - MACHINE ROUND Healthcare Patient Hospital Readmission Rate (Within 30 Days).

Problem: Identify patients discharged who got readmitted within 30 days of discharge.

```
WITH PatientAdmissions AS (
    SELECT
        admission_id,
        patient_id,
        discharge_date,
        LEAD(admission_date) OVER (
            PARTITION BY patient_id
            ORDER BY admission_date
        ) AS next_admission_date
    FROM hospital_stays
)
SELECT
    patient_id,
    admission_id,
    discharge_date,
    next_admission_date,
    (next_admission_date - discharge_date) AS days_to_readmit
FROM PatientAdmissions
WHERE next_admission_date IS NOT NULL
    AND (next_admission_date - discharge_date) <= 30;
```

Q90 - MACHINE ROUND Continuous Temperature Anomaly Detector (3 Consecutive Spikes).

Problem: Detect IoT sensors reporting temperature > 100°C for 3 or more consecutive readings.

```
WITH ThresholdCheck AS (
    SELECT
        sensor_id,
        reading_time,
        temperature,
        CASE WHEN temperature > 100 THEN 1 ELSE 0 END AS is_spike
    FROM sensor_telemetry
),
StreakGroups AS (
    SELECT
        sensor_id,
        reading_time,
        temperature,
        is_spike,
        ROW_NUMBER() OVER (PARTITION BY sensor_id ORDER BY reading_time) -
        ROW_NUMBER() OVER (PARTITION BY sensor_id, is_spike ORDER BY reading_time) AS grp
    FROM ThresholdCheck
)
SELECT sensor_id, MIN(reading_time) AS spike_start, MAX(reading_time) AS spike_end, COUNT(*) AS
spike_count
FROM StreakGroups
WHERE is_spike = 1
GROUP BY sensor_id, grp
HAVING COUNT(*) >= 3;
```

Q91 - MACHINE ROUND Hotel Room Dynamic Pricing: Median Rate by Season and Room Type.

Problem: Calculate the exact 50th percentile (Median) booking price per room type across seasonal periods.

```
SELECT
    season,
    room_type,
    PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY nightly_rate) AS median_nightly_rate
FROM bookings
GROUP BY season, room_type;
```

Q92 - MACHINE ROUND High-Concurrency Task Queue Worker Implementation using SKIP LOCKED.

Problem: Atomically claim the top 5 highest-priority pending jobs for processing without table lock deadlocks.

```
-- Atomic Job Lock and Update Transaction
WITH SelectedJobs AS (
    SELECT job_id
    FROM background_job_queue
    WHERE status = 'PENDING'
    ORDER BY priority DESC, created_at ASC
    LIMIT 5
    FOR UPDATE SKIP LOCKED
)
UPDATE background_job_queue q
SET status = 'PROCESSING',
    locked_by_worker = 'worker_node_42',
    locked_at = NOW()
FROM SelectedJobs s
WHERE q.job_id = s.job_id
RETURNING q.job_id, q.payload, q.priority;
```

Q93 - MACHINE ROUND Currency Conversion at Historical Daily Exchange Rates.

Problem: Convert transactions into USD using the most recent available exchange rate on or before the transaction timestamp.

```
SELECT
    t.tx_id,
    t.tx_date,
    t.currency,
    t.amount,
    t.amount * r.rate_to_usd AS amount_in_usd
FROM international_transactions t
CROSS JOIN LATERAL (
    SELECT rate_to_usd
    FROM exchange_rates e
    WHERE e.currency = t.currency
        AND e.rate_date <= t.tx_date
    ORDER BY e.rate_date DESC
    LIMIT 1
) r;
```

Q94 - MACHINE ROUND Social Network Mutual Friends Recommendation.

Problem: Suggest new friendships for User A based on having at least 2 mutual friends.

```
WITH Friends AS (
    SELECT user_id, friend_id FROM friendships
    UNION
    SELECT friend_id AS user_id, user_id AS friend_id FROM friendships
)
SELECT
    f1.user_id,
    f2.friend_id AS suggested_friend,
    COUNT(*) AS mutual_friend_count
FROM Friends f1
JOIN Friends f2 ON f1.friend_id = f2.user_id
WHERE f1.user_id != f2.friend_id
    AND NOT EXISTS (
        SELECT 1 FROM Friends f3
        WHERE f3.user_id = f1.user_id AND f3.friend_id = f2.friend_id
    )
GROUP BY f1.user_id, f2.friend_id
HAVING COUNT(*) >= 2;
```

Q95 - MACHINE ROUND Flight Booking Overlapping Date Collision Check.

Problem: Detect overlapping room bookings for the same room id.

```
SELECT
    b1.booking_id AS booking_1,
    b2.booking_id AS booking_2,
    b1.room_id,
    b1.check_in,
    b1.check_out
FROM hotel_reservations b1
JOIN hotel_reservations b2
    ON b1.room_id = b2.room_id
    AND b1.booking_id < b2.booking_id
    AND b1.check_in < b2.check_out
    AND b1.check_out > b2.check_in;
```

Q96 - MACHINE ROUND Calculate User Sessionization (30-Minute Inactivity Window).

Problem: Group clickstream events into user sessions where a lapse of >30 minutes indicates a new session.

```
WITH EventLag AS (
    SELECT
        user_id,
        event_time,
        CASE WHEN event_time - LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) > INTERVAL
'30 minutes'
            OR LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) IS NULL
            THEN 1 ELSE 0 END AS is_new_session
    FROM clickstream
),
SessionIDs AS (
    SELECT
        user_id,
        event_time,
        SUM(is_new_session) OVER (PARTITION BY user_id ORDER BY event_time) AS session_id
    FROM EventLag
)
SELECT user_id, session_id, MIN(event_time) AS session_start, MAX(event_time) AS session_end, COUNT(*) AS
page_views
FROM SessionIDs
GROUP BY user_id, session_id;
```

Q97 - MACHINE ROUND Supply Chain Bill of Materials (BOM) Total Cost Calculation.

Problem: Calculate the cumulative manufacturing cost of a finished product by recursively rolling up all subcomponents and raw materials.

```
WITH RECURSIVE BomTree AS (
    SELECT parent_part_id, component_part_id, quantity, quantity AS total_qty
    FROM bom_components
    WHERE parent_part_id = 'PRODUCT_X'
    UNION ALL
    SELECT b.parent_part_id, b.component_part_id, b.quantity, t.total_qty * b.quantity
    FROM bom_components b
    JOIN BomTree t ON b.parent_part_id = t.component_part_id
)
SELECT
    t.component_part_id,
    SUM(t.total_qty) AS required_units,
    p.unit_cost,
    SUM(t.total_qty * p.unit_cost) AS total_component_cost
FROM BomTree t
JOIN parts_catalog p ON t.component_part_id = p.part_id
GROUP BY t.component_part_id, p.unit_cost;
```

Q98 - MACHINE ROUND Stream Data Gap Filling with GENERATE_SERIES.

Problem: Ensure zero-sales days are not omitted from sales dashboards across a 30-day window.

```
WITH DateRange AS (
    SELECT GENERATE_SERIES('2026-07-01'::DATE, '2026-07-30'::DATE, '1 day'::INTERVAL)::DATE AS
calendar_date
)
SELECT
    d.calendar_date,
    COALESCE(SUM(s.amount), 0) AS daily_revenue
FROM DateRange d
LEFT JOIN sales s ON d.calendar_date = s.sale_date
GROUP BY d.calendar_date
ORDER BY d.calendar_date;
```

Q99 - MACHINE ROUND UPSERT / Merge Implementation with Conflict Handling.

Problem: Atomic insert or increment of user daily metric counter.

```
INSERT INTO user_daily_metrics (user_id, metric_date, api_calls, error_count)
VALUES (1001, '2026-08-14', 1, 0)
ON CONFLICT (user_id, metric_date)
DO UPDATE SET
    api_calls = user_daily_metrics.api_calls + EXCLUDED.api_calls,
    error_count = user_daily_metrics.error_count + EXCLUDED.error_count,
    updated_at = NOW();
```

Q100 - MACHINE ROUND High-Performance Real-Time Leaderboard with Tie-Breaker Logic.

Problem: Rank gaming players by total score (DESC); in event of a tie, rank player who achieved the score earlier higher.

```
SELECT
    player_id,
    score,
    achieved_timestamp,
    ROW_NUMBER() OVER (
        ORDER BY score DESC, achieved_timestamp ASC
    ) AS leaderboard_position
FROM player_leaderboard
LIMIT 100;
```